Honours Programme Delft
Report - Fault3r

Author: Tom Cajot
Supervisor: Mottaqiallah Taouil

1 Introduction

The goal of this project is to build a hardware extension on top of a RISC-V based SoC, capable of injecting one or more instructions into a running program - either once or a specified number of times, and triggered at a chosen program counter (PC) value or timing. The resulting tool, Fault3r, is evaluated with a fault injection attack on an AES128 implementation running on the core, demonstrating that the encryption key can be recovered.

The remainder of this report is organised as follows. Section 2 provides background on the Ibex core and on AES and its fault injection scheme used in the evaluation. Section 3 describes the design and implementation of Fault3r. Section 4 presents the evaluation, first on a simple adder program, then on the AES128 differential fault analysis (DFA) attack, and finally through a jump and link (jal) based fault to a modified memory region allowing multi-instruction fault payloads. Section 5 concludes.

2 Background

The RISC-V Core

Several cores were considered for this project. The CV32E40P [2] and its security-focused fork CV32E40S [6], both four-stage RISC-V cores, were initially provided by the supervisor. However, after persistent toolchain issues that were only diagnosed later in the project, both were set aside in favour of a fresh start with the Ibex core [5]. Ibex was selected for three reasons: it is well documented, it is written in SystemVerilog, and a simple SoC with a memory and UART was available with the core providing the ideal starting point for the project. Additionally, it is marketed as a production-quality core. Figure 1 provides a block diagram of the ibex core.

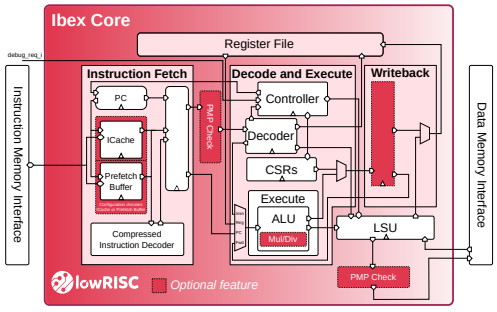


Figure 1: Block Diagram of the Ibex RISC-V Core

The core was configured as RV32IC with a 2-stage pipeline (IF and ID/EX). Most optional features were left disabled - writeback stage, instruction cache, PMP, branch predictor, and SecureIbex - resulting in a minimal baseline configuration. This configuration was chosen to avoid the additional sources of complexity and potential error that a more advanced setup would have introduced.

One aspect of the Ibex architecture is particularly relevant to this work: the core communicates with instruction memory through a simple request/grant/-valid protocol. It asserts an address on `instr_addr_o` together with a request, the memory acknowledges with `instr_gnt_i`, and the corresponding instruction word is later returned on `instr_rdata_i`. This interface is the natural injection point exploited.

AES Encryption & DFA Attack

AES is a symmetric block cipher that operates on 16-byte blocks, internally represented as a 4×4 matrix of bytes referred to as the *state*. Encryption consists of an initial round-key k addition followed by a number of rounds n, each combining four operations on the state. AES128, AES192, and AES256 are implementations that differ only in key length and round count (10, 12, and 14 rounds respectively); this work focuses on AES128. The four per-round operations are:

1. **SubBytes**: each byte of the state is substituted via a fixed lookup table (the S-box), introducing non-linearity.
2. **ShiftRows**: each row of the state is cyclically shifted left by its row index (0, 1, 2, 3 positions).
3. **MixColumns**: each column of the state is multiplied by a fixed matrix over $GF(2^8)$. This step is omitted in the final round.
4. **AddRoundKey**: the state is XORed with a round-specific key, derived from the original key through a key expansion procedure.

A more detailed explanation, including key expansion and the inverse operations used for decryption, is given in [4] and a complete implementation can be found at the following repository [3].

An outline of the differential fault analysis (DFA) attack used is given here; the original attack is due to Piret and Quisquater [7], and a great walkthrough can be found in [8].

The attack uses the same single-byte fault model as [8], inserting a fault in row r within a column of the state at the input of the ninth - and last - MixColumns operation. Let x denote the state byte that, in an unfaulted run, arrives at the input of the round-10 SubBytes at the position under analysis, and let c be the corresponding ciphertext byte. Without a fault, $c = S(x) \oplus k_{10}$, where k_{10} is the round-10 key byte at that position. With the fault, MixColumns spreads f across the four rows of column n as $(m_0 \cdot f, m_1 \cdot f, m_2 \cdot f, m_3 \cdot f)$, where $(m_0, \ldots, m_3)$ is the r-th column of the MixColumns matrix and f is the XOR difference between the unfaulted and faulted state byte. The faulted ciphertext byte at the same position therefore becomes $c' = S(x \oplus m_r \cdot f) \oplus k_{10}$.

Combining these two expressions and applying the inverse S-box on both sides eliminates x and places a constraint on k_{10}:

$$S^{-1}(c \oplus k_{10}) \ \oplus \ S^{-1}(c' \oplus k_{10}) \ = \ m_r \cdot f. \qquad (1)$$

Since we know r and f, the right-hand side is a known constant, and equation (1) is checked against every guess of $k_{10} \in \{0, \ldots, 255\}$. The values for which it holds are kept as candidates for that key byte.

Equation (1) is stated per byte, but a single injected fault actually constrains *four* bytes of k_{10} simultaneously: MixColumns propagates the input fault to all four rows of column n, and ShiftRows in round 10 then scatters those four bytes to four fixed ciphertext positions (for example, column 0 of the state ends up at ciphertext bytes $0, 13, 10, 7$). This gives four instances of equation (1) that share the same f.

A single faulted ciphertext is not enough to find a unique set of key bytes for a column of the round key. We therefore fault all the bytes in the column and intersect the candidate sets. Repeating this across all four columns recovers the full round-10 key k_{10}, from which the original key follows by running the AES-128 key schedule in reverse.

The requirements imposed on Fault3r by this attack are therefore: the ability to **corrupt a chosen single byte of the AES state** at a chosen point in the encryption, and the ability to **retrieve** the resulting **ciphertext** for analysis.

3 Fault3r - Fault Injection Extension

Fault3r is implemented at the top-level RTL file of the Ibex core, sitting between the Ibex core's instruction memory interface and the instruction memory itself. This placement keeps the Ibex source untouched. The mechanism consists of sequential and combinational logic that monitors `instr_addr_o` and

instr_gnt_i and swaps instr_rdata_i when a configured trigger condition is met.

Fault3r is configurable from the testbench through the following parameters:

- bit FAULT: master enable. When deasserted, Fault3r is transparent and the core behaves exactly as without the extension.
- int unsigned FAULT_ADDR: the instruction address at which the fault is to be triggered.
- int unsigned FAULT_INSTR: the 32-bit instruction word that will replace the original at FAULT_ADDR when the trigger fires.
- byte FAULT_ADDR_COUNT: the number of times FAULT_ADDR must be visited before the fault becomes active.
- bit FAULT_REPEAT: if asserted, the fault fires on every visit to FAULT_ADDR from the FAULT_ADDR_COUNT-th onwards; if deasserted, it fires only on the FAULT_ADDR_COUNT-th visit and not afterwards.

The code for this can be found on the Github repository [1] of this project, together with a detailed documentation outlining the process of running programs on the core and using fault3r.

4 Evaluation

A Simple Adder

The adder program of Listing 1 was used as a test case for Fault3r. The objective was to inject a fault that changes int a = 4 to int a = 1, and to verify the change by observing both the instruction memory interface and the relevant register in simulation.

```
int main(int argc, char *argv[]) {
    int a = 4;
    int b = 7;
    int c = a + b;
    return 0;
}
```

Listing 1: Simple Adder Program

Compiling the program with the toolchain [1] produces, alongside the memory image that contains the program running on the core, a .dump that exposes the encoded instructions and their addresses. Its relevant part is shown in Listing 2: at address 0x212c the instruction word 0x00400793 is fetched, decoded as li a5, 4. Replacing this word with 0x00100793 (li a5, 1) should change the value loaded into a5 from 4 to 1 thereby changing the variable assignment as intended, and propagating through the subsequent addition.

```
00002118 <main>:
    2118: fd010113          addi  sp,sp,-48
    211c: 02812623          sw  s0,44(sp)
    2120: 03010413          addi  s0,sp,48
    2124: fca42e23          sw  a0,-36(s0)
    2128: fcb42c23          sw  a1,-40(s0)
    212c: 00400793          li  a5,4
    2130: fef42623          sw  a5,-20(s0)
    2134: 00700793          li  a5,7
    2138: fef42423          sw  a5,-24(s0)
    213c: fec42703          lw  a4,-20(s0)
    2140: fe842783          lw  a5,-24(s0)
    2144: 00f707b3          add a5,a4,a5
    2148: fef42223          sw  a5,-28(s0)
    214c: 00000793          li  a5,0
    2150: 00078513          mv  a0,a5
    2154: 02c12403          lw  s0,44(sp)
    2158: 03010113          addi  sp,sp,48
    215c: 00008067          ret
```

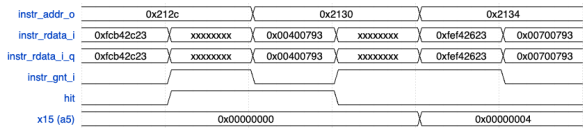
Listing 2: Adder .dump file part

The simulation was run with Fault3r configured as follows:

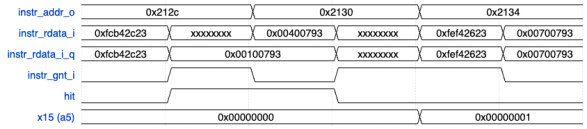
- FAULT = 1'b1
- FAULT_ADDR = 32'h0000212c
- FAULT_INSTR = 32'h00100793
- FAULT_ADDR_COUNT = 8'd0
- FAULT_REPEAT = 1'b0

The signals instr_addr_o, instr_rdata_i, instr_rdata_i_q, instr_gnt_i, hit, and register a5 (x15) were monitored. Figure 2a shows the baseline run, while Figure 2b shows the run with Fault3r enabled. In the faulted run, upon reaching the targeted address with instr_gnt_i asserted, indicating that the core is ready to accept instruction data, the instruction word on instr_rdata_i_q is overridden by the configured FAULT_INSTR. At address 0x212c, the observed value becomes 0x00100793 rather than the original 0x00400793, causing a5 to hold 1

instead of 4. This confirms that Fault3r correctly intercepts and replaces the targeted instruction, and that the modification propagates through the pipeline as expected.



(a) Unfaulted Adder



(b) Faulted Adder

Figure 2: Simulation waveforms for the adder, both faulted and unfaulted.

Breaking AES

To perform the attack, a simple C implementation of AES-128 was adapted from [3]. Custom logging functions were added to output the resulting ciphertexts to predefined memory locations, making it easier to analyze the results and its intermediate states.

$$\begin{bmatrix} 54 & 4F & 4E & 20 \\ 77 & 6E & 69 & 54 \\ 6F & 65 & 6E & 77 \\ 20 & 20 & 65 & 6F \end{bmatrix}$$

Plaintext

$$\begin{bmatrix} 54 & 73 & 20 & 67 \\ 68 & 20 & 4B & 20 \\ 61 & 6D & 75 & 46 \\ 74 & 79 & 6E & 75 \end{bmatrix}$$

Key

$$\begin{bmatrix} 29 & 57 & 40 & 1A \\ C3 & 14 & 22 & 02 \\ 50 & 20 & 99 & D7 \\ 5F & F6 & B3 & 3A \end{bmatrix}$$

Ciphertext

$$\begin{bmatrix} 80 & 57 & 40 & 1A \\ C3 & 14 & 22 & CD \\ 50 & 20 & C8 & D7 \\ 5F & 92 & B3 & 3A \end{bmatrix}$$

Byte 0 Faulted Ciphertext

Table 1: Plaintext - Key - Ciphertext - Fault (in hex)

After running the reference encryption (using the key, plaintext, and ciphertext pair specified in Table 1), analysis of the simulation and `.dump` file allowed the instructions targeting individual bytes of the state during the ninth ShiftRows operation, immediately preceding the final MixColumns, to be located.

These instructions were faulted individually by overwriting them with `0x00`, one at a time, while allowing the encryption to continue normally. Each such fault zeroes one byte of the round-9 MixColumns input, so the fault value f from Section 2 corresponds to the original (unfaulted) value of that byte. Faulting all four bytes of a single column in turn produced four faulted ciphertexts targeting that column, and repeating this process across all four columns yielded the full set of 16 faulted ciphertexts. One example is shown in Table 1; the remaining ciphertexts are available in the GitHub repository [1] alongside their corresponding `FAULT_INSTR` and `FAULT_ADDR` values to allow reproduction of the experiment.

A Python script (also in the repository) implements the recovery from Section 2: for each column it applies equation (1) at the four ciphertext positions affected by that column's faults, sweeps $k_{10} \in \{0, \ldots, 255\}$ at each position, and intersects the candidate sets across the four faulted ciphertexts of the column, which gives the full round-10 key k_{10}.

From k_{10}, the original key k_0 is recovered by running the AES-128 key schedule in reverse. The k recovered this way matches the key of Table 1, confirming a successful end-to-end attack.

Multi-Instruction Faults

Replacing a single instruction restricted `Fault3r` to faults representable within a single 32-bit RISC-V instruction. To overcome this, a Python script was developed to inject arbitrary multi-instruction payloads.

The script automates the redirection of program execution through the following steps:

1. **Memory Scanning:** It searches the program's memory for a zero-filled region large enough to hold the custom payload plus a return jump.

2. **Payload Insertion:** It writes the user-supplied instruction sequence into this identified region.

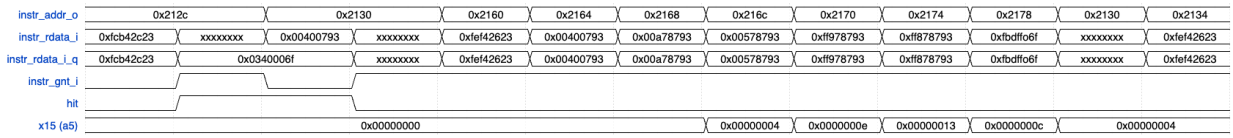


Figure 3: Multi-instruction fault waveforms. The injected JAL at 0x212c diverts execution to the payload region (0x2160–0x2174), after which the original control flow resumes transparently.

3. **Return Linkage:** It appends a `jal` command to the payload, pointing back to `FAULT_ADDR` + 4 to ensure execution resumes immediately after the injected fault without corrupting the core's return address register.

4. **Redirection:** The script generates a `jal` (jump and link) instruction targeting the new payload.

By setting `FAULT_INSTR` to this generated jump, `Fault3r` diverts the CPU to execute the complex sequence before seamlessly returning to the original control flow. This allows for significantly more sophisticated fault simulations than standard single-instruction swaps.

This was tested on the adder program from Section 4 with the payload shown in Listing 3. The script located a suitable zero-filled region starting at `0x2160`, placed the payload at `0x2160` up to `0x2174`, and returned `0x0340006f`, corresponding to `jal x0, 0x2160`. Fault3r was configured with `FAULT_ADDR = 32'h0000212c` and `FAULT_INSTR = 32'h0340006f`; all other parameters were as in the previous subsection.

```
00400793        # addi a5, zero, 4
00a78793        # addi a5, a5, 10
00578793        # addi a5, a5, 5
ff978793        # addi a5, a5, -7
ff878793        # addi a5, a5, -8
fbdff06f        # jal  zero, 0x2130
```

Listing 3: Multi-instruction fault payload injected at `0x2160`.

The same signals as in Section 4 were monitored. The waveform (Figure 3) shows the execution diverging from the unfaulted run at `0x212c`: control jumps to `0x2160`, the four `addi` instructions are executed in turn - `a5` taking the values 4, e, 13, e, and 4 in hex - before the final `jal` returns control to `0x2130`. From this point on the program proceeds as in the unfaulted run, confirming that the JAL-based mechanism correctly executes a multi-instruction payload and resumes the original control flow.

5 Conclusion and Reflections

This project set out to build a configurable hardware fault-injection extension for a RISC-V SoC, and to use it to demonstrate a real cryptographic attack. Both objectives were met: Fault3r supports parametrizable single-instruction faults triggered on a chosen program counter and visit count, and was extended through a JAL-based mechanism to support arbitrary multi-instruction fault payloads. Using Fault3r, a DFA attack was successfully executed against an AES128 implementation running on the Ibex core, recovering the secret key from 16 faulted ciphertexts.

Several extensions to Fault3r are worth noting. An intermediate version triggered faults on a cycle count rather than a PC match; exposing both trigger types as configurable parameters would broaden the range of expressible attacks. The current implementation is also limited to one fault site per run; supporting multiple independent sites would enable multi-fault campaigns.

The largest source of lost time was a toolchain misconfiguration that went undiagnosed for months: the makefile invoked the RISC-V compiler with an extension not enabled in the CV32E40S core used. The program compiled and produced a memory image, but the instructions could not execute correctly - a build problem hiding as a fault-injection problem. The lesson is to check the compiler's target ISA against the core's configuration before debugging anything else.

Beyond the specific outcome, the project provided significant exposure to digital design and computer architecture, particularly to the practical aspects of bringing up a non-trivial RTL project: makefiles, TCL scripts, simulation flow, and the relationship between high-level C code and the instructions ultimately executed by the core. These skills have already proven valuable in subsequent academic and professional work in the field.

References

[1] Tom Cajot. *fault3r: Main Project Repo*. 2026. URL: https://github.com/tomcajot/fault3r.

[2] OpenHW Group. *CORE-V CV32E40P RISC-V Core*. Accessed: 2026-05-03. GitHub. 2024. URL: https://github.com/openhwgroup/cv32e40p.

[3] Intel Corporation. *TinyCrypt: A library of cryptographic algorithms with a focus on small, simple implementation*. GitHub repository; accessed 3 May 2026. 2024. URL: https://github.com/intel/tinycrypt.

[4] Avinash Kak. *Lecture 8: AES: The Advanced Encryption Standard*. Lecture Notes on "Computer and Network Security", Purdue University. Updated February 5, 2026. Accessed: 2026-05-03. 2026. URL: https://engineering.purdue.edu/kak/compsec/NewLectures/Lecture8.pdf.

[5] lowRISC. *Ibex: A Small 32-Bit RISC-V CPU Core*. Accessed: 2026-01-18. GitHub. 2025. URL: https://github.com/lowRISC/ibex.

[6] OpenHW Group. *CV32E40S: 4-stage, secure RISC-V core*. Accessed: 2026-01-18. GitHub. 2025. URL: https://github.com/openhwgroup/cv32e40s.

[7] Gilles Piret and Jean-Jacques Quisquater. "A Differential Fault Attack Technique against SPN Structures, with Application to the AES and Khazad". In: *Cryptographic Hardware and Embedded Systems - CHES 2003*. Vol. 2779. Lecture Notes in Computer Science. Springer, 2003, pp. 77–88. DOI: 10.1007/978-3-540-45238-6_7.

[8] Philippe Teuwen and Charles Hubain. *Differential Fault Analysis on White-box AES Implementations*. Quarkslab Blog. Dec. 2016. URL: https://blog.quarkslab.com/differential-fault-analysis-on-white-box-aes-implementations.html.